Implementation Document

For

PlaceIIT

Version 1.0

Prepared by

Group 2: **Group Name: Soft_warriors**

Devarsh Mehta	240650	devarshms24@iitk.ac.in
Abhirup Vasa	241147	vasa24@iitk.ac.in
Anoop Muthavarapu	240672	anoopm24@iitk.ac.in
Duvvuri Jyothi Samanvith	240380	jyothids24@iitk.ac.in
Joy Vardhan Yalla	241192	yallaj24@iitk.ac.in
Krish Mesariya	240560	kmesariya24@iitk.ac.in
Tumati Harsha Vardhan Reddy	241107	tumati24@iitk.ac.in
Kartik S Basant	240527	kartiksb24@iitk.ac.in
Pravesh Thalor	240791	pravesh24@iitk.ac.in
Sumath Rengasami V	241054	sumathr24@iitk.ac.in

Course: CS253

Mentor TA: Seetaramayya Lavu

Date: 27 March 2026

CONTENTS II
REVISIONS II

1 IMPLEMENTATION DETAILS 4

2 CODEBASE 9

3 COMPLETENESS 12

APPENDIX A - GROUP LOG 23

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Initial Draft (v1.0)	Abhirup Vasa Kartik S Basant Pravesh Thalor Joy Vardhan Yalla Tumati Harsha Vardhan Devarsh Mehta Sumath Rengasami V Krish Mesariya Anoop Muthavarapu Duvvuri Jyothi Samanvith	First Draft	27/03/26

1 Implementation Details

PlaceIIT is structured around a three-layer architecture comprising a Presentation Layer, an Application Layer, and a Data Layer, each with a clear delineated set of responsibilities that interact through well-defined interfaces. This separation of concerns is particularly well suited to a real-time, high-concurrency placement coordination system, where changes to interview queue state must propagate instantly to multiple simultaneous browser sessions without disrupting the independent logic of the API or the persistence mechanisms of the database.

The architecture naturally accommodates the role-based workflows that PlaceIIT enforces: Students interact exclusively with a read-oriented portal, Company Coordinators (CoCos) manage shortlists and interview schedules for their assigned companies, and Higher-Level Administrators (APCs) exercise full system control including bulk data operations, analytics, and notification dispatch. The following subsections document the specific technologies selected for each layer, along with the rationale for their adoption in the context of PlaceIIT's functional and non-functional requirements.

1.1 Presentation Layer (Project Unit)

The Presentation Layer constitutes the client-side interface of PlaceIIT, responsible for rendering the three distinct user portals and for communicating live state changes to end users in real time. The following technologies form the foundation of this layer.

Programming Languages :

- **HTML5 and CSS3 :** For structuring and styling the web pages.
- **JavaScript :** The primary language for interactive behavior in the browser.

Framework and Build Tool :

- **React 18 :** Component-based UI library with a Virtual DOM that efficiently handles frequent re-renders during live queue updates.
- **Vite :** Modern build tool with Hot Module Replacement (HMR), replacing Create React App for faster development cycles.

Libraries and Tools :

- **React Router DOM :** Manages navigation across the Student, CoCo, and APC portals within the single-page application.
- **TailwindCSS v4 :** Utility-first CSS framework enabling a consistent design system across all three user portals.

- **Radix UI / shadcn/ui :** Accessible, unstyled component primitives used for Modals, Dropdowns, and interactive UI elements.
- **Lucide React :** Provides a consistent icon set across the interface.
- **Socket.io Client :** Enables real-time bidirectional communication between the browser and server for live queue tracking.
- **Framer Motion :** provides smooth micro-animations to improve the perceived responsiveness of queue status updates (we use the new 'motion' package name for version 12).
- **Recharts :** Used for placement statistics visualisation on the APC Admin dashboard.

Justification of Choices:

- **React 18** was selected as the primary UI library because its Virtual DOM diffing algorithm minimises the number of actual DOM mutations triggered by queue state changes, a critical property when dozens of simultaneous users may observe the same interview queue updating at sub-second intervals; no comparable alternative in the JavaScript ecosystem combines this rendering efficiency with a mature ecosystem of accessible component libraries.
- **Vite** was adopted in place of Create React App because its native ES-module-based development server and Hot Module Replacement mechanism drastically reduce feedback cycles during frontend development, which directly accelerated iteration on the three distinct portal interfaces.
- **TailwindCSS** was chosen to maintain visual and behavioural consistency across the Student, CoCo, and APC portals: by constraining design decisions to a shared utility vocabulary, the framework prevents the divergence in spacing, typography, and colour usage that commonly arises when multiple developers contribute independently to different interface sections.
- **Socket.io** Client was a non-negotiable dependency at the presentation layer because the live queue tracking feature, which reflects instantaneous changes in interview call-up status for students awaiting their turn, cannot be implemented satisfactorily through polling-based approaches; the library's managed WebSocket abstraction provides the necessary real-time event delivery while gracefully falling back to long-polling in restricted network environments.

1.2 Application Layer (Logic Unit)

The Application Layer encapsulates all server-side logic for PlaceIIT, including request routing, authentication enforcement, queue state management, file processing, and outbound notification dispatch. The following technologies constitute this layer.

Programming Language:

- **JavaScript (ES6+, Node.js runtime) :** For all server-side development, maintaining a unified language environment across the full stack.

Framework:

- **Express.js :** Minimalistic and flexible web framework for building the RESTful API; bridges the frontend and the database.
- **Node.js :** Non-blocking, event-driven JavaScript runtime enabling the server to handle many simultaneous connections efficiently.

Libraries and Tools:

- **Mongoose :** ODM library that enforces schemas on MongoDB documents and manages relationships between Users, Students, and Companies.
- **JWT + bcryptjs :** Used together for secure user authentication and password hashing, supporting Role-Based Access Control (RBAC) across the three user roles.
- **Socket.io :** Synchronises live interview queue state across all connected clients in real time.
- **Multer :** Handles file uploads including student resumes and bulk Excel sheets.
- **XLSX :** Parses Excel files to support bulk upload of student shortlists and company data by APC administrators.
- **Nodemailer :** Sends OTPs and automated placement notifications via the IITK Webmail server.

Justification of Choices:

- **Node.js** was selected as the server-side runtime because its event-driven, non-blocking I/O architecture is inherently well-matched to PlaceIIT's concurrency profile: during peak placement activity, hundreds of students, multiple CoCos, and APC administrators may maintain simultaneous persistent connections to receive live queue updates, a workload that would saturate a thread-per-connection model but is handled efficiently by Node's single-threaded event loop. The use of Node.js also preserves a homogeneous JavaScript environment across the entire stack, reducing context-switching overhead for developers and enabling the direct reuse of validation logic and data transformation utilities between client and server modules.
- **Express.js** was preferred over more opinionated frameworks such as NestJS because its deliberately minimal core imposes no structural constraints on the API design, allowing the team to compose only the middleware required by PlaceIIT's specific routing topology, authentication pipeline, and file-upload handling without carrying the overhead of unused framework features.

- **Mongoose** was adopted as the application's data access layer because, although MongoDB is schema-flexible by nature, PlaceIIT's data integrity guarantees, particularly the enforcement of mandatory fields on Student and Company documents and the validation of enumerated placement status values, require schema-level constraints that Mongoose provides through its model definition and built-in validation hooks, while also simplifying error propagation by converting low-level MongoDB driver errors into structured, actionable exceptions.
- **Socket.io** was a non-negotiable server-side dependency because the real-time bidirectional synchronisation of interview queue state, the mechanism by which a CoCo's action of calling the next candidate is instantly reflected on every connected student's screen, constitutes a core functional requirement of the system that cannot be deferred to periodic HTTP polling without introducing unacceptable latency and user confusion during time-sensitive interview sessions.

1.3 Data Layer (Data Unit)

The Data Layer is responsible for the persistent storage and retrieval of all PlaceIIT entities, including student records, company profiles, interview round definitions, queue snapshots, and notification logs. The following technologies underpin this layer.

Database System:

- **MongoDB :** NoSQL, document-oriented database storing JSON-like records; accommodates varying interview data structures (e.g., companies with differing round formats) without requiring rigid schema migrations.

Database Interaction Library:

- **Mongoose :** Serves as the interface between the Node.js application and MongoDB, providing schema definitions, document validation, and pre/post middleware hooks.

Deployment Tools:

- **MongoDB Atlas :** Managed cloud hosting for MongoDB offering high availability, automated daily backups, and real-time monitoring; reduces operational overhead during critical placement periods.

Justification of Choices:

- MongoDB was selected in preference to a relational database because the interview data managed by PlaceIIT is structurally heterogeneous: different recruiting companies define different numbers of interview rounds, attach varying sets of metadata to each round, and may alter their round structure mid-season, all of which would necessitate costly schema migrations in a fixed-schema relational system but are trivially accommodated by MongoDB's document model without downtime or data restructuring. Additionally, the high-frequency read and write

pattern of the live queue system, where queue documents are updated and broadcast multiple times per minute during active interview days, benefits from MongoDB's document-level locking and in-memory write buffering, which sustain the throughput needed without contention.

- MongoDB Atlas was chosen as the hosting platform because the managed infrastructure it provides, encompassing automatic failover, point-in-time restore, and continuous backup, removes the operational burden of database administration from a student development team and provides the reliability guarantees that are essential during the placement season, a period in which data loss or prolonged downtime would have direct, irreversible consequences for student placement outcomes.

1.4 Authentication and Authorization

PlaceIIT employs JSON Web Tokens (JWT) as the exclusive mechanism for both authentication and authorization across all three user roles. Upon submission of login credentials, the Application Layer verifies the supplied password against the bcrypt-hashed value stored in the database; a successful verification results in the issuance of a signed JWT that encodes the user's unique identifier and their assigned role, which may be one of student, coco, or admin. Every subsequent API request originating from the client must carry this token in the Authorization header, where a server-side middleware intercepts and validates it before the request is permitted to proceed to any route handler; this validation step enforces Role-Based Access Control (RBAC) by ensuring that students can only access student-scoped endpoints, CoCos are restricted to operations on their assigned companies, and APCs are granted full system access. Tokens are issued with a defined expiry duration, and any request carrying an expired or absent token is rejected with an appropriate HTTP error code, redirecting the client to the Login page and thereby enforcing a zero-trust model in which no persistent session state is maintained on the server.

1.5 Version Control

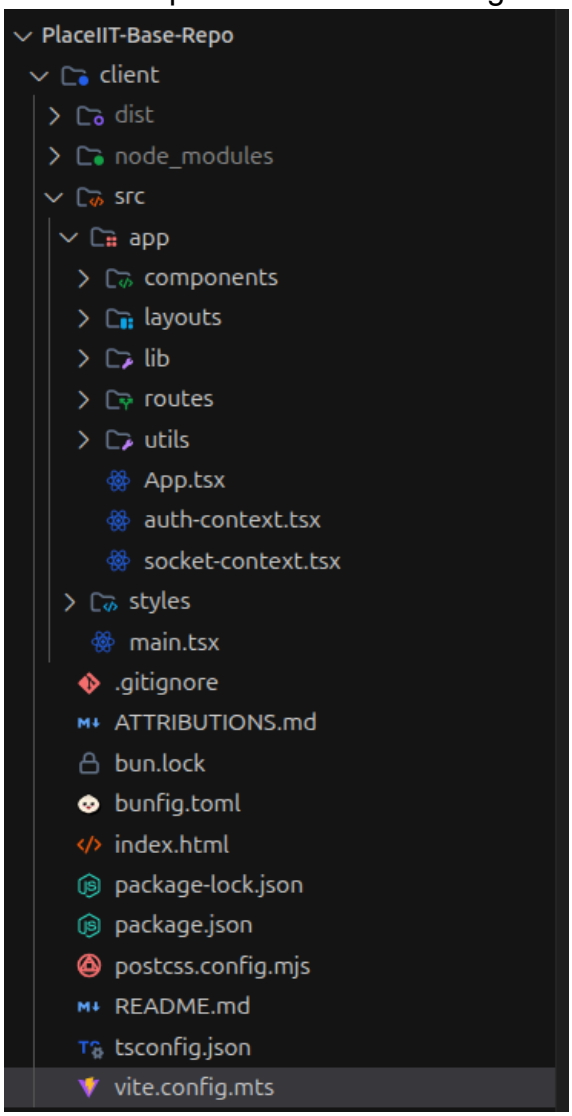
- **Git** was adopted as the version control system for PlaceIIT, providing a comprehensive change history that enables the development team to audit, revert, and trace every modification made to both the frontend and backend codebases across the duration of the project.
- **GitHub** was used as the centralized remote repository platform, offering collaboration features such as pull request workflows, code review tooling, and branch protection rules that enforced a disciplined integration process.

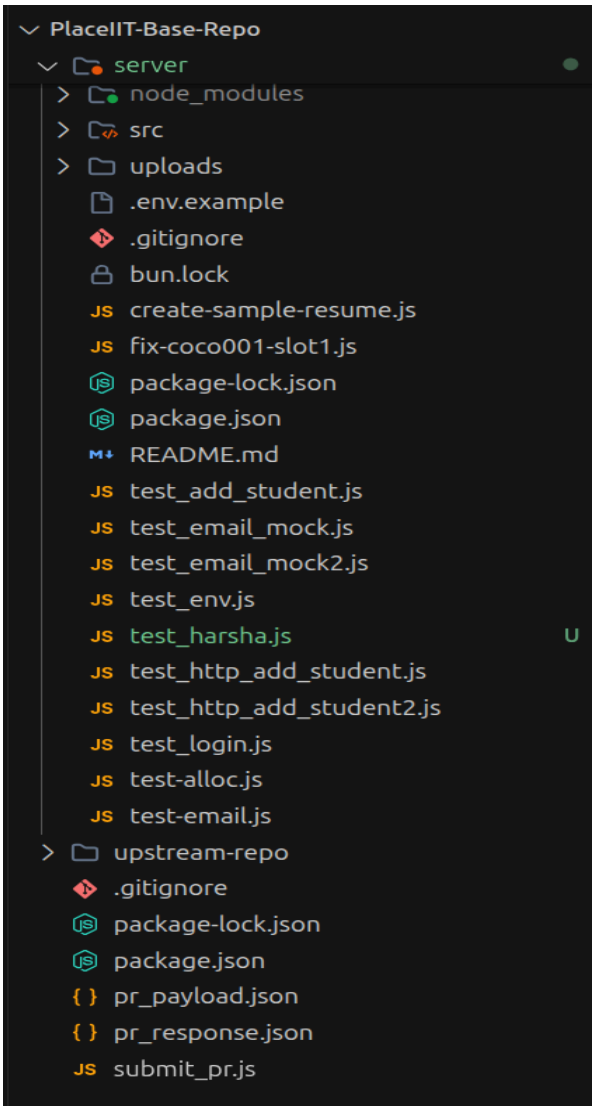
The team maintained dedicated branches for individual features and bug fixes, merging them into the main integration branch exclusively through pull requests that required at least one peer review, a practice that ensured no untested or unreviewed code was introduced into the shared codebase. This branching strategy allowed parallel development across the Presentation and Application layers without interference, and the complete pull request history serves as an auditable record of design decisions and implementation choices made throughout the project lifecycle.

2 Codebase

We have made a repository on GitHub through which we can store and collaborate on the source code of this project PlaceIIT.

We have implemented the following file structure in the repository :





The PlaceIIT repository is organized into two primary top-level directories, ensuring a strict separation of concerns between the frontend interface and the backend application logic:

- **client/ :** Houses the frontend implementation built utilizing Vite, React 18, and TypeScript.
- **server/ :** Contains the backend server environment developed with Node.js, Express, and MongoDB.

Client Architecture (client/src/app/)

- **App.tsx :** The root component responsible for initializing the application and managing router configurations.
- **auth-context.tsx :** Encapsulates the authentication context, including login states and session token management.
- **socket-context.tsx :** Provides the Socket.IO context required for real-time bidirectional data synchronization.
- **components/ :** A library of approximately 78 reusable UI components maintaining design consistency.
- **routes/ :** Contains approximately 24 page-level route components for portal navigation.
- **layouts/ :** Structural wrappers including shared headers and sidebars for the user portals.
- **lib/ :** Stores shared libraries and helper configurations used across the frontend.
- **utils/ :** General-purpose utility functions for data transformation and validation.
- **styles/ :** Global CSS definitions and Tailwind framework configurations.

Server Architecture (server/src/)

- **server.js :** Primary entry point for the Express application and the Socket.IO server initialization.
- **config/ :** Manages database connection logic and environmental configuration parameters.
- **models/ :** Defines approximately 12 Mongoose schemas enforcing data integrity for all entities.
- **controllers/ :** Implements request handling logic for the API layer (~6 functional modules).
- **routes/ :** Definitions for all RESTful API endpoints and their associated handlers.
- **services/ :** Encapsulates the core business logic layer (~8 specialized service files).
- **middlewares/ :** Handles authentication, role-based validation, and error interceptors.
- **sockets/ :** Contains event handlers for the Socket.IO real-time communication engine.
- **utils/ :** Helper utilities for backend data processing and formatting.
- **seed*.js :** Scripts utilized for database environment seeding and initial data population.

Workflow and Implementation Guidelines

- **API Execution Path :** Requests flow sequentially through routes, controllers, and services before interacting with Mongoose models.
- **Frontend Navigation Path :** URL routes render specific components supported by integrated authentication and socket contexts.
- **Real-time Synchronization :** Managed via Socket.IO, bridging the client-side socket context with the server-side event handlers.

3 Completeness

PlaceIIT is a real-time campus placement coordination platform developed for the Indian Institute of Technology Kanpur (IITK) community. It is designed as a centralized portal that supports the full lifecycle of an on-campus placement drive: from student registration and resume submission to live interview queue management, coordinator communication, and administrative oversight. The platform serves three distinct user roles — Students, Company Coordinators (CoCos), and Higher-Level Administrators (APCs) — each interacting with a tailored interface that exposes only the functionality relevant to their responsibilities. The system's defining capability is its real-time queue engine, which propagates interview state changes instantaneously across all connected sessions, enabling students to monitor their queue position, CoCos to manage panel throughput, and APCs to observe placement progress system-wide.

Features Implemented:

I. User Authentication and Access Control

A. Superuser-Controlled Account Creation and Login

The system does not permit direct user sign-ups. Instead, account creation is strictly restricted to the primary "Superuser" , who is responsible for creating and adding all other users to the system. The Superuser inputs their email IDs and other necessary details, upon which the newly added users automatically receive their unique User ID and Password directly via their registered email. Users then authenticate using these provided credentials, and upon successful verification, the system automatically redirects them to their role-specific home page or dashboard, granting seamless access to authorized functionalities without requiring manual role selection.

B. Forgot Password with OTP

A password recovery workflow has been implemented in three stages:

- The user enters their registered institute roll number.
- A one-time password (OTP) is dispatched to that address via the IITK Webmail server, and the user verifies the OTP within the portal.
- After successful OTP verification, the user sets and confirms a new password, whereupon they are redirected to the Login page.

C. Role-Based Access Control

The system enforces strict Role-Based Access Control (RBAC) across three user roles. Students are restricted to personal interview management and read-oriented views. CoCos operate exclusively within the scope of their assigned companies, performing queue management and student communication tasks. APCs hold full system oversight privileges including bulk data operations, coordinator allocation,

II. Student-Facing Functionality

A. Complete and Edit Profile

Upon first login, students are required to complete their profile by entering the mandatory fields: Emergency Contact. An Edit Profile button subsequently re-enables all fields for modification, allowing students to maintain accurate personal information throughout the placement season.

B. Consolidated Interview Status View

A consolidated overview panel displays the student's interview status across all shortlisted companies in a single view. Each entry carries a real-time status indicator drawn from the set: Upcoming, In Queue, In Interview, Completed, or Rejected. This view eliminates the need to navigate individual company cards to assess overall placement progress.

C. My Companies Dashboard

A dynamic grid or list-based dashboard presents all companies for which the student has been shortlisted according to the interview schedule (Day/Slot). Each company card displays the company name, current active round, scheduled day and slot, venue, serving as a centralized hub for quick reference.

D. Company Status Indicators and Actions

Each company card is equipped with real-time status tags (In Queue, Upcoming, Completed, and others) and contextual interactive actions. A Join Queue button is rendered exclusively when the relevant interview round is active, which sends request to coco of that company to accept. Also, there is the option to cancel the request to join the queue.

E. Real-time Queue Position Display

For queues the student has joined, the system displays the student's exact real-time position (e.g., "Waiting: 5th in line") along with the count of candidates ahead. This information allows students to estimate remaining wait times and make informed decisions about managing overlapping interview commitments.

F. View Queue Length of Other Companies

For companies the student is shortlisted for but has not yet joined, the system displays the current queue length and the number of candidates waiting. This visibility enables the student to make informed decisions about when to join a queue without committing prematurely.

G. Exit Queue

A dedicated Exit Queue action is available on each active company card. Upon confirmation, the student is removed from the queue in real time, their status is updated across all connected sessions, and their availability for other concurrent interviews is restored.

H. View and Join Walk-in Opportunities

A dedicated Walk-in section, accessible via a navigation button that auto-scrolls to the relevant area, lists all currently active walk-in drives. Each walk-in card displays the company name, round, timing, slot, venue, current queue length, and status. Students can join a walk-in queue directly from the card, with the system handling real-time queue insertion and dispatching appropriate notifications.

I. Search and Filter Companies

A search bar enables students to query their shortlisted companies by name or round. Complementary filter controls allow sorting or segmenting results by interview round or day, improving navigation efficiency for students managing multiple concurrent shortlists.

J. Real-time Notifications

The notification system delivers updates through two parallel channels: a bell icon in the navigation bar with an unread count badge, and non-intrusive toast pop-ups rendered at the bottom of the screen. Notifications cover queue position changes, interview call-up alerts, venue changes, walk-in announcements, predefined coordinator messages (e.g., "Report to venue immediately" or "Bring resume"), and system-level broadcasts such as maintenance alerts. Each notification carries a category label (Alert or Info), a timestamp, and a read/unread

state. Users can mark individual notifications as read, mark all as read, or clear all notifications.

K. Contact Us and Query System

A query submission form allows students to submit support requests by entering a subject and a detailed message body. Upon submission, a success toast confirms delivery to the Assistant Placement Coordinators (APCs) . A My Queries section displays the student's historical submissions along with any responses or feedback received, providing a traceable communication record.

L. Quick Help and FAQ

A Quick Help section lists frequently asked questions and resolutions to common student issues, such as problems with queue joining or notification delivery. This self-service resource reduces the volume of direct support queries directed at SPO staff.

M. Important Contacts Directory

An interactive directory displays key contacts in card format, including Assistant Placement Coordinators (APCs), Company Coordinators (CoCos), and Technical Support personnel, with each card showing name, designation, email address, and phone number.

N. Immediate Action "Red Flag" System

To facilitate urgent communication, a "Flagging" mechanism will be introduced for Company Coordinators (CoCos). If a coordinator requires a candidate's immediate presence, they can "flag" the student's profile. This will instantly trigger a prominent "Red Flag" alert on the student's dashboard, explicitly showing the venue details and instructing them to report immediately, superseding standard queue notifications.

III. Company Coordinator (CoCo) Functionality

A. View Assigned Companies

The CoCo interface lists all companies assigned to the coordinator, filterable by day and slot. Each entry displays the company name, venue, scheduled date, and the count of associated students, providing a consolidated overview of the coordinator's responsibilities for each placement day.

B. Manage Walk-in Availability

CoCos can activate or deactivate walk-in mode for any of their assigned companies via a single toggle action. Activation automatically adds the company to the walk-in lists visible to all eligible students and dispatches real-time notifications, ensuring timely student participation.

C. Manage Interview Panels

CoCos can add, edit, or remove interview panels for their assigned companies. Each panel entry displays real-time details including panel location, ongoing round, panel members, the student currently under interview, and idle or active status. CoCos can assign students from the waiting queue to idle panels directly, optimising panel utilisation and minimising idle time.

D. Round-based Queue and Status Management

The queue management interface uses a column-based layout with one column per interview round (e.g., Round 1, Round 2). Top-level counters display the number of active students per round. Within each column, students are grouped by status: Yet to Interview, In Queue (with a sub-counter of waiting candidates), In Interview, and Completed. CoCos can add students individually or in bulk, update individual statuses, and assign students to specific panels from this interface.

E. Student List and Live Tracking per Slot

A comprehensive student list for a specific day and slot displays each student's name, roll number, contact number, emergency contact, and real-time status or location, including indicators such as "In queue for another company" or "Currently interviewing." This live tracking capability allows CoCos to assess student availability before calling them for an interview and avoid disruptions caused by concurrent commitments.

F. Search and Filter Students

A search bar allows CoCos to locate students by name or roll number. Complementary filters control narrow results by round and status (Yet to Interview, Waiting, In Interview, or Completed), facilitating rapid identification and management of specific candidates during active interview sessions.

G. Send Predefined Notifications to Students

CoCos can select and dispatch predefined template messages to individual students or to groups. Available templates include "Report to the interview venue immediately," "Bring a physical copy of your resume," and "Venue change." The use

of standardised templates ensures rapid, consistent communication without requiring CoCos to compose messages under time pressure.

H. View Student Details

CoCos can access detailed student profiles comprising full contact information, emergency contact details, and placement history. This information supports informed decision-making during the interview process and facilitates coordinator-student communication.

I. CoCo Profile and Statistics

The CoCo profile page displays editable personal details (Full Name, User ID, Email, Phone Number, Department) accessible via an Edit Profile button. A statistics summary card presents metrics including the total number of companies assigned, students managed, and the date the coordinator joined the portal. A linked list of assigned companies provides direct navigation to each company's detailed view.

IV. Higher-Level Administrator (APC / Placement Admin) Functionality

A. System-wide Overview and Statistics Dashboard

The APC dashboard provides an aggregate overview of the placement system, displaying key metrics including the total number of registered students, active companies, and assigned coordinators. This high-level view enables APCs to monitor placement season progress at a glance, and also provides the option to send system-wide notifications.

B. Manage Companies

APCs can add new companies or edit existing company records, with configurable fields including company name, job role, day, slot, and venue. A searchable and filterable card-based company list supports rapid location of specific entries. Additionally, APCs can upload Excel files containing company information, which are automatically parsed by the XLSX library and integrated into the database, populating company details, venues, and schedules without manual entry.

C. Upload and Process Student Shortlist Excel

The system supports bulk upload of student shortlist data via Excel files. Upon upload, the file is parsed and processed automatically to assign the listed

students to their respective companies, eliminating manual data entry and reducing the risk of transcription errors during the onboarding of new placement drives.

D. Coordinator Requirement Upload and Auto-Allocation

APCs can upload coordinator requirement data, whereupon the system performs automated and randomised allocation of CoCos to companies and slots based on availability and workload, ensuring fair distribution. Manual override and reassignment options are available to resolve specific conflicts or preferences identified by the APC, including the ability to assign more than one CoCo to a single company.

E. Manage Coordinators (CoCos)

APCs can add new CoCo accounts by entering details including name, email address, and phone number. A summary card view presents all active coordinators with their contact details and assigned company counts. A detailed schedule view shows coordinator-wise assignments broken down by company, day, and slot, enabling the APC to detect and resolve scheduling conflicts.

F. Global Search across Students, CoCos, and Companies

A global search facility allows APCs to query across all three entity types: students (searchable by Name, Roll Number, or Email), CoCos, and companies. Filter controls refine search results, supporting the administrative monitoring and auditing tasks that APCs perform throughout the placement season.

G. View Company and Interview Details

An expanded company detail view presents the full interview configuration for a selected company, including the scheduled day and slot, assigned CoCo, venue, and a summarised student status overview. Additional controls allow the APC to search and filter the shortlisted student list by placement status and academic branch for granular analysis.

H. Manage APCs (Super Admin Exclusive)

The primary or "God" APC account includes an exclusive "APCs" section in the navigation bar, providing top-level oversight of administrative access. This section allows the primary administrator to create and manage Sub-Admins (APCs). Features include an "Add APC" functionality to onboard new sub-administrators, a search bar to locate APCs by name or ID, and an "APC Summary" dashboard. The summary displays individual cards for each APC, showing their email, phone number, and a direct action button to delete or revoke their administrative access.

I. Notifications Management

APCs receive and manage system updates through a dedicated Notifications panel, accessible via a bell icon (with unread count badges) in the top navigation bar. This panel provides real-time alerts regarding placement activities, such as notifications when specific companies initiate walk-in interviews at designated venues. Administrators can search through notifications, mark individual or all alerts as read, and clear the notification history to maintain an organized feed.

J. Administrator Profile and Security Settings

The portal includes a dedicated profile management page where administrators can view their basic information, including Full Name, Email Address, Phone Number, Institute ID, and specific Role. Furthermore, a "Security Settings" section provides tools for administrators to manage their account security preferences, prominently featuring a "Change Password" functionality and tracking when the password was last modified. An additional "Account Information" block confirms the user's account type and active status.

V. Cross-role and System-wide Functionality

A. Centralised State-based Queue Engine

A centralized queue management engine maintains the state of every active interview queue, tracking each student’s position, the overall queue length, and individual status transitions in real time. State updates are synchronised across all connected user sessions via Socket.io, ensuring that every role observes a consistent, accurate view of queue state with no discrepancies during parallel interview operations.

B. Real-time Data Synchronisation

Changes to interview statuses, queue compositions, and notification events are propagated instantly to all relevant connected clients. The system maintains a single source of truth for all mutable placement state, eliminating the inconsistencies that arise when multiple coordinators and students interact with the same queue simultaneously.

C. Role-specific Dashboards and Navigation

Each user role is served a fully customised dashboard and navigation bar that surfaces only the data, views, and actions relevant to that role. Students see a queue-centric interface; CoCos see panel management and student tracking tools;

APCs see aggregate statistics and data management controls. This role-aware interface design reduces cognitive load and prevents accidental access to out-of-scope functionality.

D. Comprehensive Notification Delivery System

The notification framework supports in-app delivery via a persistent bell-icon badge, real-time push delivery through Socket.io events, durable storage of notification records with categorisation (Alert or Info), and deep-linked actionable entries. The system ensures that all three user roles receive timely, relevant updates throughout active placement operations, supporting coordinated workflows across geographically dispersed participants.

Non-Functional Requirements met :

A. Security

IITK email-based student registration, combined with JWT authentication and bcrypt password hashing, ensures that only authorised IITK community members access the platform. Role-Based Access Control enforced at every API endpoint prevents privilege escalation across the three user roles. OTP-based password recovery adds an additional verification layer against unauthorized account takeover.

B. Real-time Performance

The Socket.io-based queue synchronisation engine delivers sub-second state propagation to all connected clients during active interview sessions. The Node.js event-driven runtime sustains the concurrent connection load generated by simultaneous student, CoCo, and APC sessions without thread-contention bottlenecks, maintaining responsive performance during peak placement activity.

C. Reliability and Data Integrity

MongoDB Atlas provides automated daily backups, point-in-time restore, and managed high-availability failover, ensuring that student and placement data is never permanently lost due to infrastructure failure. Mongoose schema validation enforces data integrity at the application layer, rejecting malformed records before they reach persistent storage.

D. Usability

Role-specific dashboards, real-time status indicators, and contextual action controls reduce the navigation burden for all user categories. The TailwindCSS-based design system maintains visual consistency across all three portals. Accessible component primitives from Radix UI ensure that interactive elements such as modals, dropdowns, and form controls conform to WCAG accessibility standards.

E. Communication Protocols

HTTP/HTTPS is used for all RESTful API communication between the client and server. The WebSocket protocol, managed through Socket.io with automatic long-polling fallback, handles all real-time bidirectional queue and notification updates. IITK Webmail is used as the outbound mail relay for OTP delivery and automated placement notification emails via Nodemailer.

F. Memory and Storage

The application maintains a dedicated MongoDB Atlas cluster for all persistent data, including user records, company configurations, queue state snapshots, resume file metadata, and notification logs. Resume files are stored as server-side uploads managed by Multer, with file paths and timestamps recorded in the database.

G. Scalability and Modularity

The backend follows a layered service-oriented architecture with clearly separated concerns — controllers handle request routing, services encapsulate business logic, and models define data schemas. This modular decomposition allows individual components such as the queue engine, notification service, or allocation service to be modified, tested, or replaced independently without cascading changes across the codebase. The Socket.io room-based event architecture (company:<id>, user:<id>) ensures that real-time updates are scoped to relevant subscribers, preventing unnecessary broadcast fan-out as simultaneous placement sessions scale.

H. Input Validation and Error Handling

A centralized error-handling middleware intercepts all unhandled exceptions and returns structured JSON error responses, exposing stack traces only in development mode to prevent information leakage in production. File upload endpoints enforce MIME-type whitelisting and size limits (10 MB cap for Excel uploads, PDF-only for resumes) through Multer middleware configuration, preventing arbitrary file injection. Role-based validation on the forgot-password flow ensures OTP-based resets are only permitted for accounts matching the originating portal role, closing cross-role account takeover vectors.

I. Maintainability and Developer Experience

A centralised API service layer on the frontend (api.ts) abstracts all HTTP communication behind typed, domain-specific method groups (authApi, studentApi, cocoApi, adminApi), eliminating scattered fetch calls and ensuring consistent token injection and error propagation. Environment-driven configuration through .env files

and Vite's loadEnv enables seamless switching between development and deployment targets without code modification. Automated Excel-based bulk data ingestion for students, companies, coordinators, and shortlists reduces manual administrative overhead and minimises human data-entry errors during large-scale placement drives.

Possible Features for Future Updates :

A. Customisable Interview Rounds

The system will include provisions to support an extended, customizable number of interview rounds (such as Group Discussions, multiple Technical rounds, and HR rounds).
But right now as much as we have discussed with SPO teams three rounds are enough to support all the interviews practically. This enhancement will accommodate the highly varied and specific recruitment processes of different visiting companies.

B. Student Applications and Priority Preferences

Students will be provided with the option to actively apply for visiting companies directly through the portal and explicitly fill out their priority preferences. This feature will seamlessly automate shortlist generation and help coordinators better manage overlapping interview schedules.

C. In-Platform Resume Verification

A dedicated workflow will be implemented for placement administrators to review, verify, and approve uploaded student resumes. This will eliminate the need for external tools, establishing PlaceIIT as a truly comprehensive, end-to-end centralized platform.

D. View Final Selection Results

The capability to display final placement outcomes, such as "Selected" or "Rejected," will be added directly to the student and company dashboards.They can also select the company they need to select at the end of a day. This addition will bring full transparency to the placement lifecycle, allowing students to track their final results within this single website.

E. Built-In Online Assessments (OA)

A centralized testing module will be developed to conduct preliminary online assessments, including coding rounds and aptitude tests. This will allow companies to evaluate candidates directly within the PlaceIIT ecosystem prior to the interview stages.

F. Built-In Online Interviews

Built-in video conferencing and virtual interview rooms will be integrated to seamlessly support remote and hybrid recruitment drives. This will allow coordinators to manage online interviews with the exact same efficiency and centralized tracking as offline venues.

Appendix A - Group Log

Team Coordination & Communication :

- Established a dedicated WhatsApp group to facilitate real-time communication, quick decision-making, and transparent coordination among all team members throughout the project lifecycle.
- Created and maintained a private GitHub repository with a structured branching strategy (feature branches, pull requests, and code reviews) to ensure version control, traceability, and collaborative development.
- Adopted a clear task ownership model where each member was assigned a primary module while also contributing to cross-functional areas, ensuring end-to-end understanding of both frontend and backend systems.
- Maintained a shared project tracker (to-do list with deadlines and ownership tags) updated after every meeting to keep all members aligned on deliverables and timelines.

What Sets Our Group Apart :

- **Proactive code review culture :** All pull requests were reviewed by at least one other member before merging, improving code quality and shared understanding of the codebase.
- **Structured meeting protocol :** Every meeting began with a fixed agenda and concluded with documented action items and assigned owners — minutes were recorded and shared via the group chat within 30 minutes of each session.
- **Early integration approach :** Instead of merging all work at the end, we adopted incremental integration checkpoints, which significantly reduced last-minute conflicts and debugging overhead.
- **Consistent documentation habit :** Inline code comments, README updates, and architectural notes were maintained throughout development — not added as an afterthought — making the codebase self-explanatory and submission-ready.
- **Balanced workload distribution :** Tasks were divided by competency and interest rather than arbitrarily, resulting in higher quality deliverables and a more motivated team.

Meeting Log :

S.No	Date	Venue	Summary of the Meeting
1	06 January 2026	Hall 2	Discussed newly proposed ideas from different domains.
2	08 January 2026	Starbucks	Discussed the feasibility of alternative project ideas, including carpooling systems.

S.No	Date	Venue	Summary of the Meeting
3	10 January 2026	RM408 & RM Ground Floor	Presented shortlisted ideas to Prof. Indranil Saha; confirmed "PlaceIIT" as the final project.
4	11 January 2026	Hall 2	Finalized the project idea and submitted the initial high-level system description.
5	15 January 2026	Cafe Coffee Day	Finalized UI design and discussed detailed requirements; divided into three sub-teams for SRS drafting.
6	18-22 Jan 2026	Hall 2	Collaborative sessions focused on drafting Requirement Documentation.
7	22 January 2026	RM Ground Floor	Met with Mentor TA Seetaramayya Lavu to finalize SRS document after modifications.
8	24 January 2026	Hall 2	Incorporated final modifications and submitted the completed SRS document.
9	27 January 2026	Online	Divided into teams and distributed workload for UI design and architecture.
10	30 January 2026	Cafe Coffee Day	Finalized classes, design patterns, and overview of system algorithms.
11	03 February 2026	Hall 2	Validated progress; finalized UI, state, and sequence diagrams.
12	05 February 2026	RM Ground Floor	Discussed document progress with the Teaching Assistant and resolved technical doubts.
13	06 February 2026	Hall 2	Final meeting to consolidate work and prepare the final Software Design Document draft.
14	11 February 2026	Hall 2	Defined foundational architecture and visual theme; set up the project repository.

S.No	Date	Venue	Summary of the Meeting
15	03 March 2026	Online	Finalized the database schema and established a unified communication framework (REST API, state management).
16	18 March 2026	Hall 2	Structured progress review; identified blockers and recalibrated priorities for the final sprint.
17	24 March 2026	Online	Completed full-stack integration; resolved merge conflicts and refined UI/UX.
18	27 March 2026	Hall 2	Finalized Implementation Document and performed end-to-end testing before submission.